We Lost 25 Hours of Scheduled `claude -p` Jobs to One Silent Hang

We Lost 25 Hours of Scheduled `claude -p` Jobs to One Silent Hang

Here's the harness that makes yours fail loud, fast, and once.

The operations manual from a 15-runner fleet's incident log — not the quickstart. For anyone running an unattended, scheduled `claude -p` that something, or someone, depends on.

On 2026-07-11, one `claude -p` run on our 15-runner fleet hung — no error, no timeout, no exit. Because launchd won't launch a second copy of a job it still thinks is running, that one stuck process **silently swallowed every job scheduled behind it for 25 hours.** We didn't get an alert. We got a quiet day that looked like any other. `[BEYOND-LLM: receipt — a dated row in this fleet's operating log]`

That was the week's first lesson. The second was worse. The tripwire we'd built to shout when a rail goes dark was **itself dead** — retired from the scheduler *and* `bash -n`-broken by a stray apostrophe, at the same time — so the one thing watching the runners was, itself, running dark. `[BEYOND-LLM: receipt]`

This guide is three things from that week: the ~50-line hardened runner, an auditor that scores *your* runners against the same six failures (point `preflight-audit.sh` at your fleet, get a gap list), and the dated incident log behind every guard. On the next page we run that auditor on our *own* fleet — and some of it still fails. The harness makes an unattended `claude -p` fail *loud* (a dead run pages you), *fast* (a hang is killed in minutes, not hours), and *once* (a double-fire can't overlap). **What it will not do is cap your dollar spend** — that is a workspace spend limit, and we point you to where it lives — but it *will* make sure you hear about a stuck or duplicated run in minutes, not a day later. We name that limit up front on purpose: trust a runner's guardrails exactly as far as the person who tells you where they end.

The answer, in six lines

- **Headless Claude on a schedule is a *wrapper* problem, not a Claude problem.** Claude runs one shot and exits. Every way it fails unattended lives in the ~50 lines around `claude -p`, and in the scheduler. Get the wrapper right and the failures below can't happen to you unseen. `[COMMODITY — the baseline agrees; this framing is not our edge]`
- **Six ways an unattended runner dies in the dark, and they are countable:** it **hangs**, **burns tokens silently**, **double-fires**, **bleeds state**, **blocks on a permission prompt**, or the fleet **drifts**. Name all six and you check them off. `[BEYOND-LLM: decision-tool]`
- **The load-bearing pair: a single-flight lock is worthless without a timeout.** A hung run holds the lock forever, so every later fire skips silently — you've rebuilt the outage you were preventing. We learned this at **25 hours dark**. `[BEYOND-LLM: receipt]`
- **Your scheduler picks your poison.** launchd and systemd *refuse to overlap* a job with itself → a hang becomes a **silent skip** (a timeout saves you). cron *allows overlap* → a slow run becomes a **double-fire** (a lock saves you). The same guard is load-bearing on one platform and irrelevant on the other. `[BEYOND-LLM: tested-procedure]`
- **Make it loud, then guard the loudness.** A dead runner must *page you*, not fail quietly to the invoice. And your tripwire needs its own tripwire — ours was unloaded *and* syntactically broken for three days and nothing caught it. `[BEYOND-LLM: receipt]`
- **Honest scope, so you don't feel cheated:** this makes a runaway **loud fast** and **caps model spend** — it is **not** a hard dollar cap. The hard dollar ceiling is a workspace spend limit (two clicks in the Console) or a deterministic boundary hook. We name it; we don't re-sell it. `[BEYOND-LLM: proprietary-data]`

Not for you if a hang costs you nothing. If your scheduled jobs are throwaway experiments and a silent skip is free, close the tab — you don't need this. This is the manual for the runner that something, or someone, is counting on: a client deliverable, a product's users, a report a human reads. The qualifier is blast radius beyond yourself, not team size. **And not for you if you'd rather an LLM just assemble the mechanics for you** — it can, and it'll be *correct*; §1 shows you exactly that, then concedes it. Buy this for the receipts, the interactions *between* the guards, and the *cut-list* — what to remove so the runner fails **once** — the part no clean model can derive, because it never ran the fleet. `[BEYOND-LLM: honest qualification]`

Before the theory: our own runners, scored — and they aren't clean

Here is the thing a quickstart can't show you, dated today. We ran the six-death auditor (§9, bundled) across the **11 runners installed on our box** — the growth engine *and* the three that researched, built, and graded this guide (7 of the 11 call `claude -p`; the table shows those). It maps where real runners skimp, which beats any hand-wave that "you should add guards":

`[BEYOND-LLM: proprietary-data — real fleet audit,` receipts/2026-07-27-fleet-audit.txt `, run 2026-07-27]`

`claude -p` runner	job	D1 timeout	D2 pages-on-fail	D3 lock	D4 isolation	D5 perm	D6 model pinned
`chesky-autopost`	posts content	✅ `alarm 900`	✅ 🚩 on empty	⚠️ launchd-only	⚠️ own cwd	✅	✅ opus
`chesky-drift-heal`	heals selectors	✅ `alarm 300`	⚠️ success-only	⚠️ launchd-only	⚠️ DOM-as-data	✅	✅ sonnet
`guide-build`	built this guide	❌ **none**	❌ log-only	✅ `mkdir` +trap	✅ `mktemp -d`	✅	✅ `$MODEL`
`guide-grade`	graded this guide	❌ **none**	❌ log-only	✅ `mkdir` +trap	⚠️ composed	✅	✅ `$MODEL`
`guide-propose`	proposed this guide	❌ **none**	❌ log-only	✅ `mkdir` +trap	⚠️ composed	✅	✅ `$MODEL`
`consult-twin`	board advice	❌ *by design*	❌ *(foreground)*	❌	❌	✅	✅ opus
`studio-engagement`	staleness sweep	❌ **none**	❌	⚠️ PID-file race	⚠️	✅	❌ *unpinned*

Read the ❌ column and wince with us. Only 2 of our 7 `claude -p` runners wrap the model call in a hard timeout — and 4 of the 5 that don't run *unattended*, including the runners that built this guide. The fleet that got burned at 25 hours *still* hasn't hardened its own guide-factory rail.

That's not a confession for color. It's the point of Death 6 (drift) and the reason the auditor exists: **guards you add once rot unevenly, and you can't see the rot without a tool that scores every runner.** The fleet even ships *both* half-right locks — atomic `mkdir` (leaks on `kill -9`) and a PID-file (races) — and no runner combined them until §4. `[BEYOND-LLM: proprietary-data]`

This table is also the answer to *"is this stack-gated to your one weird setup?"* — no: the six failures recur across **two independent fleets and eleven runners**, on jobs that share nothing but the shape `schedule → claude -p → exit`. The spellings are Mac-flavored; the deaths are not.

Page 1 — the hardened runner template

This is the whole answer as one file. Each guard carries the failure it prevents and the real script it came from, and it now folds in three day-2 hardenings (kill-escalation, a lock that survives `kill -9`, and a missing-state assert). Copy it, replace the prompt and the allow-list, and the six deaths can't happen to you silently. `[BEYOND-LLM: decision-tool — a synthesis of the live fleet runners; every mechanic here was executed in this build, output in` receipts/2026-07-22-demo-output.txt `+` 2026-07-27-variant-output.txt `]`

```
#!/usr/bin/env bash
# hardened claude -p runner — the six guards, annotated with the failure each
        stops.
set -uo pipefail

# --- config ------------------------------------------------------------------
JOB="repo-digest"                                   # unique per runner (names the
        lock + logs)
MODEL="${JOB_MODEL:-claude-haiku-4-5}"              # GUARD 6: pin cost; env-
        overridable per runner
LOG="$HOME/claude-logs/$JOB-$(date +%Y%m%d-%H%M%S).log"; mkdir -p "$(dirname
        "$LOG")"
ALARM(){ printf '🚩 %s: %s\n' "$JOB" "$1" | your-pager-send >/dev/null 2>&1 ||
        true; }  # GUARD 2

# --- GUARD 3: single-flight lock that survives kill -9 (atomic mkdir + PID/age
        reclaim) -------
LOCK="/tmp/claude-$JOB.lock"
if mkdir "$LOCK" 2>/dev/null; then echo $$ >"$LOCK/pid"; date +%s >"$LOCK/born"
        # atomic: one winner
else
  hpid=$(cat "$LOCK/pid" 2>/dev/null); born=$(cat "$LOCK/born" 2>/dev/null || echo
        0)
  if { [ -n "$hpid" ] && ! kill -0 "$hpid" 2>/dev/null; } || [ $(( $(date +%s)-
        born )) -gt 3600 ]; then
    echo "reclaiming leaked/stale lock (holder $hpid)"; echo $$ >"$LOCK/pid"; date
        +%s >"$LOCK/born"
  else echo "another $JOB run holds the lock — skipping"; exit 0; fi
fi
trap 'rmdir "$LOCK" 2>/dev/null || rm -rf "$LOCK"' EXIT   # released even if the
        timeout kills claude
```

```
# --- GUARD 5: state isolation — compose the prompt from files; assert it; run in
        a scratch cwd
rf(){ [ -f "$1" ] && cat "$1" || echo "(MISSING: $1)"; }
PROMPT="Do today's $JOB. Use ONLY the context below; do not invent numbers.
=== STATE ===
$(rf "$HOME/state/$JOB.md")
=== YESTERDAY ===
$(rf "$HOME/state/$JOB-last.md")"
case "$PROMPT" in *"(MISSING:"*) ALARM "a state file is missing — refusing to run
        half-blind"; exit 1;; esac
cd "$(mktemp -d)" || exit 1                          # no stray ./CLAUDE.md, no
        cwd bleed

# --- GUARD 1: hard timeout that ESCALATES to SIGKILL ⊗ GUARD 2 ⊗ GUARD 4 (turn
        cap) ----------
# A SOFT signal isn't a guarantee: a wedged child can ignore SIGTERM/SIGALRM
        (tested, DEMO 8).
# coreutils `timeout -k` escalates for you; macOS has no `timeout`, so background
        + watch + KILL.
run_capped(){ local s="$1"; shift                    # $1=seconds, rest=command
  if command -v timeout >/dev/null; then timeout --signal=TERM --kill-after=30s
        "$s" "$@"; return $?; fi
  "$@" & local p=$!; ( sleep "$s"; kill -TERM "$p" 2>/dev/null; sleep 30; kill -9
        "$p" 2>/dev/null ) & local w=$!
  wait "$p" 2>/dev/null; local rc=$?; kill "$w" 2>/dev/null; return "$rc"; }
OUT="$(run_capped 900 claude -p "$PROMPT" \
        --model "$MODEL" \
        --max-turns 20 \                             # GUARD 4: hard cap on the
        agentic loop
        --output-format json \                       # so you KNOW what happened,
        headless
        --allowedTools "Read,Grep,Bash(git status:*)" \  # GUARD 4b: least
        privilege
        --permission-mode acceptEdits 2>>"$LOG")     # GUARD 2b: never block on a
        prompt
rc=$?

# --- GUARD 2: be LOUD on empty / timeout / error — never fail to the invoice ---
if [ "$rc" -ne 0 ] || [ -z "$OUT" ]; then ALARM "empty/timeout (rc=$rc) — run
        skipped"; exit 1; fi
is_error=$(printf '%s' "$OUT" | jq -r '.is_error')   # NB: is_error can be true on
        a zero exit code
cost=$(   printf '%s' "$OUT" | jq -r '.total_cost_usd')
[ "$is_error" = "false" ] || { ALARM "claude reported error (\$$cost)"; exit 1; }
printf '[%s] ok $%s\n' "$(date '+%F %H:%M')" "$cost" >> "$HOME/state/$JOB-
        heartbeat.log"
```

Then schedule it (launchd on a Mac, systemd on a server), **and point a second, dumb watchdog at `$JOB-heartbeat.log`** — because the tripwire above can't fire if the run never starts or the box is down. That out-of-run guard is the seventh line of defense, and §7 is the week we learned we needed it.

§1 — The quickstart is right about day one; this is the manual for week three `[COMMODITY, conceded]`

The docs and a dozen 2026 write-ups hand you the happy path, and they are not wrong. `claude -p "prompt"` runs non-interactive, prints, and exits with a status code you can branch on. `--output-format json` returns a parseable envelope (`is_error`, `num_turns`, `total_cost_usd`, `session_id`); `--max-turns` caps the loop; `--model` picks the tier; `--allowedTools` least-privileges it; `--permission-mode` / `--dangerously-skip-permissions` stops it blocking on approvals. "Wrap it in a `timeout`" and "add a lock" are standard advice. **A clean LLM will tell you all of this, correctly.** `[COMMODITY — verified against our own` 02-llm-baseline.md`, which independently produced every item in this paragraph]`

So we concede it and move on. None of the above is what you're paying for, because none of it is what breaks after the job has been running unattended for a month. What breaks is the *interactions* between these guards, the *scheduler-specific* failure modes, and the silent deaths that a status code never shows you. That's the rest of this guide, and it comes from a fleet that has actually run these — and buried a few.

> ***What is this fleet?*** *15+ scheduled `claude -p` runners on one machine doing real operating work: a LinkedIn content engine that generates and self-scores posts, hourly health digests texted to a human, board-advisor consults, a selector self-healer, and the three runners that researched, built, and graded this guide. Not a demo. The incidents below are dated rows in its operating log.* `[BEYOND-LLM: proprietary-data]`

The six deaths, as three pairs — "what does it take to trust a runner you're not watching?"

A system is defined by its interactions, so the guards come in pairs. Each pair answers one half of trust.

Pair 1 — Fail safe: **timeout ⊗ tripwire** (so a stall can't run dark)

§2 — Death 1: it hangs at 3 a.m. and silently eats every later run `[BEYOND-LLM: receipt]`

Context. Our LinkedIn runner fires three times a day. On 2026-07-10 one generation call to the `claude` CLI hung — no error, no output, no exit.

The artifact (`scripts/chesky-autopost.sh` , the fix comment is dated in the file):

```
# 2026-07-11: HARD TIMEOUT (15 min) on the generation call — a hung claude CLI
        blocked the rail
# for 25h (one stuck 14:30 run silently absorbed every later scheduled run;
        launchd won't
# double-launch a running label). perl-alarm because macOS has no coreutils
        `timeout`.
BUNDLE="$(perl -e 'alarm 900; exec @ARGV' -- "$CLAUDE_BIN" -p "$GENPROMPT" --model
        opus ... 2>/dev/null)"
```

What it cost us. From the operating log, 2026-07-11: *"a hung claude-CLI generation call blocked autopost ~25h (07-10→07-11) → reach fell 1,791→188 impr/wk WoW."* Twenty-five hours dark, and **every scheduled fire in that window silently did not happen** — on a paid plan. We didn't get an error. We got *nothing*, which looked exactly like a quiet day. `[P — dated operating-log row,` receipts/2026-07-22-artifacts.md `artifact 6]`

The non-obvious why — and where the baseline is quietly wrong. A clean LLM (and every quickstart) will tell you launchd "won't start a second copy of the same job — that's free double-fire protection," and list it as a *pure positive*. It is not. **That exact property is what turned a single hang into a 25-hour outage:** because launchd refuses to launch a label that's still "running," the wedged 14:30 process meant the 18:00, 22:00, and every later fire were *silently skipped* — the scheduler thought the old one was still working. The protective feature and the catastrophic feature are the same feature. No baseline connects those two facts, because neither one is wrong on its own; only the *interaction* bites. `[BEYOND-LLM: receipt — corrects a latent, confident baseline claim]`

The fix, and its Linux equivalent. A hard wall-clock timeout around the call. On macOS there is no coreutils `timeout` , so we use the built-in `perl -e 'alarm N; exec @ARGV'` . On Linux, it's one line:

```
timeout --signal=TERM --kill-after=30s 900 claude -p "$PROMPT" ...   #
        Linux/coreutils
perl -e 'alarm 900; exec @ARGV' -- claude -p "$PROMPT" ...           # macOS, zero
        deps
```

Reuse it. We ran both this build. On this stock Mac, `command -v timeout` finds nothing — which is *why* the perl idiom exists — and `perl -e 'alarm 2; exec @ARGV' -- sleep 999` returned **rc=142 after 2s** (`SIGALRM` , 128+14), not after 999. Pick the branch your host supports (the template does it with `command -v`), set the wall-clock to ~2× your job's p99 runtime, and a wedged run now dies instead of eating tomorrow. `[BEYOND-LLM: tested-procedure —` receipts/2026-07-22-demo-output.txt `, DEMO 2 & 3]`

The day-2 variant we didn't test the first time: a *catchable* timeout isn't a timeout. `perl -e 'alarm'` raises `SIGALRM`; coreutils `timeout` raises `SIGTERM`. Both are catchable — a wedged child (or a shell wrapper that traps signals) can *ignore* them and sail past your wall clock. We proved it this build: a child that ran `trap "" TERM` then **survived `SIGTERM` and kept running**; only `kill -9` (uncatchable) ended it. `[BEYOND-LLM: tested-procedure — receipts/2026-07-27-variant-output.txt, DEMO 8]` So a hard timeout **must escalate to `SIGKILL`** (uncatchable): coreutils does it with `--kill-after=30s`; the zero-dep macOS path backgrounds the call and escalates `SIGTERM` → `SIGKILL` itself (the `run_capped` helper in the template — note that a `perl -e 'alarm'` handler *can't* do this, because `exec` resets the signal disposition in the child). A timeout that only asks nicely is a hope, not a guard. `[BEYOND-LLM: tested-procedure — receipts/2026-07-27-variant-output.txt, DEMO 8]`

§3 — Death 2 (the other half of "loud"): it burns tokens silently until the invoice `[BEYOND-LLM: receipt]`

A timeout stops a hang, but a runner can also fail by *spending*. The widely-reported shape of this pain: *"AI agents fail expensively rather than loudly — billing accumulates silently until the invoice arrives"*; threads titled *"20× max usage gone in 19 minutes."* `[E — pattern across ≥4 independent 2026 sources]` One post reports a runaway `claude -p` cron reaching a bill over $1,800 after two nights. `[S — single source, imported as reported, not verified; the *shape* is [E], the *number* is one person's claim]`

The tripwire that turns silent into loud. Our runner doesn't just log a failure — it *sends* one. Empty or failed generation pages a human:

```
if [ -z "$BUNDLE" ]; then
  printf '🚩 chesky-autopost: generation timed out/empty — queue refill skipped' |
        "$SEND" "$CHAT_ID"
fi
# ... and on a non-zero pipeline result or a detected alarm string:
if [ "$rc" = "3" ] || printf '%s' "$OUT" | grep -qiE "alarm|empty queue|publish
        failed"; then
  printf '🚩 chesky-autopost ALARM (rc=%s): %s' "$rc" "$(printf '%s' "$OUT" | tail
        -6)" | "$SEND" "$CHAT_ID"
fi
```

(`scripts/chesky-autopost.sh`, lines 52–67; phone redacted and the trailing `>/dev/null 2>&1 || true` send-guards trimmed for width; verbatim in `receipts/2026-07-22-artifacts.md`.) `[P]`

The receipt the first build owed you: a *real* JSON envelope, and why the exit code isn't enough. The buyer of the first draft asked, fairly, to see one actual `claude -p --output-format json` result parsed — not a `sleep`-stand-in. Here is one we captured live this build

(`claude 2.1.209` , `--model claude-haiku-4-5 --max-turns 1` , 2026-07-27): `[BEYOND-LLM: tested-procedure — receipts/2026-07-27-json-envelope.txt ]`

```
{"type":"result","is_error":false,"num_turns":1,"result":"ok",
 "total_cost_usd":0.0191098,"session_id":"63e0e658-...","duration_ms":1476,
 "usage":{"input_tokens":10,"cache_read_input_tokens":17098, ...}}
```

Two things a headless runner must do with that envelope. **First, branch on `is_error` , not the exit code** — Claude can return a partial "I couldn't do that" with `is_error:true` on a *zero* exit, which a naive `$?` check reads as success (DEMO 10 parses both branches). **Second, log `total_cost_usd` per run** so a spend spike is your earliest loop signal. And note the tell hiding in `usage` : a one-word prompt read **17,098 cached input tokens** — proof that a headless call is *not* context-free (Death 4). `[BEYOND-LLM: tested-procedure]`

What it cost us to learn. Before the tripwire existed, "no news" and "the runner is dead" were the same signal. The end-of-day digest states the doctrine in one line: **"A ZERO on a live rail is the headline"** (`scripts/chesky-eod-report.sh:73`). A runner you're not watching must make its *own* silence loud. `[BEYOND-LLM: receipt]`

Honest scope — read this before you trust the word "cost." The tripwire makes a runaway loud *fast*, and pinning a cheap model + `--max-turns` *caps what one run can spend*. Neither is a **hard dollar cap**. The real hard ceiling is org-level and it's commodity: in the Anthropic Console, give automation its **own workspace + API key + a monthly spend limit**, so a pathological loop physically cannot exceed the cap. `[COMMODITY — the baseline is right about this; use it]` The strongest *per-action* fuse we run is a deterministic boundary hook that denies dangerous calls before they execute — a different guide's subject. We **name** both; we don't dress up "loud + capped" as "won't burn a dollar," because that's the overclaim that earns a refund.

Pair 2 — Run clean: **single-flight lock ⊗ state isolation** (so two runs can't corrupt each other)

§4 — Death 3: two fires overlap and corrupt shared state `[BEYOND-LLM: receipt]`

The scheduler inversion — the teaching the baseline can't assemble. §2 was launchd's overlap-refusal biting you as a *silent skip*. cron is the mirror image: **cron fires on the clock no matter what**, so if run N overruns the interval, run N+1 starts *on top of it* — two `claude -p` processes writing the same state file, double-spending, racing. The guards invert with the scheduler:

Scheduler	Overlap behavior	The death	The load-bearing guard
launchd / systemd	*refuses* to run two of one label	a hang → **silent skip** of all later fires (§2)	the **timeout** (kills the hang) `[P]`
cron / k8s (no policy)	*allows* overlap	a slow run → **double-fire**, state corruption	the **single-flight lock** `[P]`

So the same advice — "add a lock" — is *load-bearing under cron and nearly irrelevant under launchd*, and "add a timeout" is *load-bearing everywhere but especially where the scheduler refuses overlap*. A generic answer can't tell you which you need because it doesn't know your scheduler. `[BEYOND-LLM: tested-procedure]`

The artifact (`scripts/guide-build.sh:20-25`) — an atomic `mkdir` lock, not a `[ -f ]` check:

```
LOCK="/tmp/chesky-guide-build.lock"
if ! mkdir "$LOCK" 2>/dev/null; then
  printf '[%s] build: another run holds the lock — exiting\n' "$(date '+%F
        %H:%M')" >>"$LOG"
  exit 0
fi
trap 'rmdir "$LOCK" 2>/dev/null' EXIT
```

Why `mkdir` and not a lockfile. `[ -f lock ] || touch lock` is *two* operations with a race window between them — two runs can both pass the test before either writes. `mkdir` is a *single atomic syscall*: exactly one racer creates the directory, everyone else gets an error. Zero dependencies, works on every OS (unlike `flock` , which — like `timeout` — isn't built in on macOS). We raced it live this build: run A `mkdir` → rc=0, run B on the same lock → rc=1, "skip." `[BEYOND-LLM: tested-procedure — DEMO 1]`

The interaction that is the whole point. A lock **without** a timeout is a trap. If a locked run hangs, its lock is never released, and every later run exits "already running" — you have *rebuilt launchd's silent-skip on top of cron*, which is the 25-hour outage again. We reproduced it in miniature this build: a held lock made runs B, C, D all skip silently, forever, until the holder was killed. **The timeout is what releases the lock** (it kills the hung run, whose `trap` cleans up). Lock and timeout are not two checklist items; they are one mechanism. `[BEYOND-LLM: tested-procedure — DEMO 4]`

The day-2 variant: the atomic lock *leaks* on `kill -9` , and the fleet's two locks are each half-right. The `trap` that `rmdir` s the lock never runs if the process is `SIGKILL` 'd (or the box loses power). Then the lock is orphaned and every later run skips forever — the outage again, lock-flavored. We proved it, then fixed it, this build. Our own fleet already contains *both* halves of

the answer and commits neither: the guide-factory runners use atomic `mkdir` (no race, but leaks on `kill -9`); `db-refresh.sh` uses a `[ -e ]`+PID-file lock (self-heals a leak by checking if the holder PID is alive, but has the race window `mkdir` avoids). **The complete lock is the hybrid** — atomic `mkdir` *and* a stored PID/birth-time so a later run can reclaim a leaked or stale lock:

```
fresh acquire:            ACQUIRED (fresh)
contend vs DEAD holder:   RECLAIM (holder pid 999999 is dead)
contend vs LIVE holder:   SKIP (holder pid 50371 alive, age 0s)
```

(`receipts/2026-07-27-variant-output.txt`, DEMO 7; the acquire function is in the Page-1 template.) `[BEYOND-LLM: tested-procedure]`

The incident that shows a runner-lock isn't always enough: shared external resources need their *own* lock. A single-flight lock guards one runner against *itself*. But two *different* runners that touch the same external resource can still collide. On 2026-07-11 our engagement rail used one browser profile (`.chrome-chesky`); **15 of 30 logged runs errored *"profile is already in use by another instance of Chromium"*** — two runners fighting over one profile, which silently regressed a downstream verification from 11 successes to 0. `[P — dated operating-log / interaction-log rows]` The per-runner lock was fine; the *shared resource* had no lock. The rule: **anything two runners share — a browser profile, a DB connection, a single API session, a working directory — needs a lock keyed to the resource, not to the runner.** `[BEYOND-LLM: receipt]`

§5 — Death 4: "with state" is a lie unless you isolate it `[BEYOND-LLM: tested-procedure]`

Headless runs are **stateless by default** and silently inherit *whatever is in the working directory and `~/.claude/`* — a stray `CLAUDE.md`, a leftover session — so "it worked in the demo" becomes "it does something different in cron" with no diff to explain it. `[E — documented foot-gun, ≥3 sources]` We just watched it happen: the live envelope in §3 read **17,098 cached input tokens for a one-word prompt** — that "context-free" call quietly carried ~17K tokens of ambient context, exactly the environment a headless run is supposed to shed. There are two honest ways to have state, and session-resume (`--resume <id>`) is the *fragile* one: a lost or expired session breaks the chain. `[COMMODITY — the baseline correctly recommends stateless-by-default]`

Our pattern — compose the prompt from files, run in a scratch cwd. This is the real "with state," and it's un-generatable because it's a *layout*, not a flag. The build runner composes one prompt from **six live state files** with a trivial helper, and generates its clean baseline in a throwaway directory so *no vault config can leak in*:

```
# condensed from scripts/guide-build.sh; line refs inline, verbatim in receipts/
rf(){ [ -f "$1" ] && cat "$1" || echo "(missing: $1)"; }         # guide-
        build.sh:30
PROMPT="... === PLAYBOOK === $(rf "$V/playbooks/guide-factory.md")
        === CRAFT === $(rf "$G/CRAFT.md") ... "                  # six $(rf ...)
        blocks, :56-77
[ -f "$DIR/promise.txt" ] && [ ! -s "$DIR/02-llm-baseline.md" ] && {  # first-
        write-wins guard, :45
  SCRATCH="$(mktemp -d)"                                         # guide-
        build.sh:47
  ( cd "$SCRATCH" && "$CLAUDE_BIN" -p "..." > "$DIR/02-llm-baseline.md" )   #
        clean cwd, no CLAUDE.md
}
```

Why inject state as text instead of letting the agent read files? A note in the sibling runner says it plainly: *"context INJECTED as text (no inline file-reads → clean JSON-only output)"* (`chesky-autopost.sh:17`). If you hand the model the state, its output is deterministic and parseable; if you let it *go find* the state, a scheduled run can wander your filesystem and its output stops being clean. We composed a real 4-file prompt live this build (1,507 chars, section markers intact) — see DEMO 5. `[BEYOND-LLM: tested-procedure — DEMO 5]`

The day-2 variant: `rf()` fails *open*, and a half-blind prompt is a wrong answer with full confidence. What happens when one of those six files is missing at 3 a.m.? `rf()` substitutes `(missing: …)` and the run proceeds — Claude gets a partial-context prompt, and its output *looks* fine. We reproduced it and added the fix: assert the composed prompt is complete before spending a token. `[BEYOND-LLM: tested-procedure —` receipts/2026-07-27-variant-output.txt`, DEMO 9]`

```
case "$PROMPT" in *"(MISSING:"*) ALARM "a state file is missing — refusing to run
        half-blind"; exit 1;; esac
```

State-bleed has a quieter twin: not the *wrong* state leaking in, but the *right* state silently missing. Guard both — isolate the cwd (`mktemp -d`) *and* assert the inputs.

Pair 3 — Stay cheap + hands-off: **model/effort policy ⊗ permission posture**

§6 — Deaths 5 & 6: it blocks on a permission prompt no one will answer, and the fleet drifts `[BEYOND-LLM: receipt + decision-tool]`

Permission posture. The single most common *silent* headless failure isn't a hang — it's a **permission denial**. With no TTY to approve a tool, an un-allowlisted action is refused and Claude returns a partial "I couldn't do that" that *looks like success*. `[COMMODITY — baseline`

`names this too]` You avoid the block one of two ways: allow-list exactly the tools the job needs (`--allowedTools`, least privilege), or, inside a sandbox, `--dangerously-skip-permissions`. The honest trade: skipping permissions is *how you stop the silent block*, and *precisely why* you then need a deterministic boundary hook in front of the runner — the flag removes the human "no," so something non-human has to keep it. Least-privilege allow-lists first; skip-permissions only where a sandbox and a hook have your back. `[BEYOND-LLM: proprietary-data]`

Model/effort policy — and drift is *measured*, not hypothetical. Five runners copy-pasted over months become five *different* `claude -p` calls. You don't have to take that on faith — the fleet-audit table on page 2 *is* the drift: six of our seven runners pin a model, one (`studio-engagement`) forgot and rides the default; two runners carry a hard timeout, five don't. The fix is a shared skeleton with env-overridable knobs, so the policy lives in one place:

```
MODEL="${GUIDE_MODEL:-opus}"      # guide-build.sh:16-17 — default high, override
        per runner
EFFORT="${GUIDE_EFFORT:-max}"     # e.g. GUIDE_MODEL=claude-haiku-4-5 for a cheap
        digest
```

The decision matrix we actually run (cost × quality × latency), so you're not guessing per job: `[BEYOND-LLM: decision-tool]`

Runner type	Model	Effort	Why
Judgement / drafting (grades a guide, writes a post)	`opus`	`max`	quality dominates; runs a few times/day; cost is noise
Consults / advisory (a board-twin opinion)	`opus`	`max`	short, high-leverage, human reads it immediately
High-frequency digest / triage / classify	`haiku`	default	fires often; cheap-and-fast beats deep; a wrong call is cheap to redo
Anything fanning out in parallel	cheapest that passes	capped `--max-turns`	parallel fan-out **burns the rate limit in minutes** `[E]`

Default high only where quality pays for itself; drop to `haiku` everywhere the job is frequent or forgiving. One skeleton, one row per runner, no drift — and the auditor (§9) tells you when a runner has drifted off it.

---

§7 — The tripwire that died: your loudness needs its own guard `[BEYOND-LLM: receipt]`

Here is the most expensive thing we know, and the one a quickstart will never tell you: **we built the loud tripwire, and then it died silently, and nothing caught it for days.** It died *two different ways* in one week — a bug in itself, and a deploy that took it down with the fleet — which is the actual lesson.

Death of the tripwire, take 1 — a bug in the alarm. The end-of-day digest (§3's "a zero is the headline" script) is our anti-silent-stall alarm. On 2026-07-14 we found it had a **bash syntax error the whole time** (operating log, verbatim in `receipts/`):

> *"the EOD tripwire (`scripts/chesky-eod-report.sh`) … had a **bash syntax error** (`bash -n` rc=2; an unbalanced `'` from `QUEUE's` on line 36, inside a `$()`-wrapped `<<'PY'` heredoc — a known bash parse gotcha) that errored it at the 07-11 19:05 fire." `[P — dated operating-log row]`*

An apostrophe in a *comment* — `QUEUE's` — inside a heredoc inside a command substitution was enough to break the alarm at parse time. The one instrument whose entire job was to shout "a rail went dark" could not itself start. The fix was a one-word comment reword; the lesson is that **a tripwire you never `bash -n` on deploy is not a tripwire.** (Add `bash -n "$0"` to a pre-commit hook; we did.)

Death of the tripwire, take 2 — a migration retired the whole fleet, watchdog included. Worse than a bug: on 2026-07-12 a scheduler migration *retired* the outward fleet's launchd jobs and loaded no replacements. The engine went dark — **and the two instruments that should have screamed (the EOD digest and the rail-watchdog) were themselves among the retired jobs.** Silent *by construction*. It stayed dark **~73 hours**, longer than the engine's entire prior live window, and the follower meter went blind too (its sampler was in the same dark fleet), so we couldn't even *see* the outage on the dashboard. `[P — dated operating-review rows, 2026-07-13→15]`

The fix — a watchdog that watches the watchers, and lives *outside* the fleet it guards. A separate, dumb, hourly job that trusts *no* rail to report its own health. Its header is the fleet's one-line post-mortem of the whole week:

```
# Two proven failure classes this week: (a) a hung generation call absorbed 25h of
        posting
# slots at rc=0; (b) a bash syntax error killed the retune for 2 days, visible
        only in a log
# nobody reads. This watchdog makes both LOUD within an hour:
#   * STALL:  a rail's process has been running longer than its max-runtime ->
        kill + alarm.
#   * SILENT: a rail's log hasn't been touched within its freshness window   ->
        alarm
```

(`scripts/chesky-rail-watchdog.sh:4–9`.) It checks two things nothing else can: is any rail's *process* older than its max runtime (kill it), and is any rail's *log* staler than its freshness window (alarm). The freshness check is `now − mtime > window` — and even *that* is portable-gotcha territory: macOS is `stat −f %m`, Linux is `stat −c %Y` (we confirmed `−c` is absent on this Mac this build; DEMO 6). `[BEYOND-LLM: receipt]`

Reuse it. Two rules came out of those two deaths. **(1) The thing that reports health cannot be the only thing that can fail** — run a watchdog *and* the commodity **dead-man's switch** (a run pings a URL *only on success*; an external service alerts when the ping doesn't arrive). `[COMMODITY, conceded — the baseline is right]` **(2) The watchdog must not be able to go dark with the fleet it watches.** Ours was co-located; a migration took both. Put your dead-man's switch on *someone else's* infrastructure (healthchecks.io, Cronitor) so it survives your whole machine — and your whole deploy — going dark. The tripwire that shares a fate with the fleet is not a tripwire.

---

§8 — The set we've hardened against — and what's deliberately out of scope

These six — hang, silent-burn, double-fire, state-bleed, permission-block, fleet-drift — are the ways an unattended `claude −p` dies *in the dark*: the failures that are silent, that a status code hides, that only surface weeks in. We present them as **reproduced, not proven exhaustive** — the receipts show each of the six is real, not that there's no seventh. So don't take "complete" on faith; try to break it. The falsifier we *didn't* test: a run that exits **zero and clean but writes a truncated or malformed artifact** — it didn't hang, didn't burn, didn't double-fire. Maybe it folds under silent-burn; maybe it's death seven. Find one that fits none of the six and it earns a row in the kit. Short of that, give your runner all six guards plus a watchdog and it fails *loud* — trust the checklist over re-enumerating from scratch. (Death 3 has the one sub-case worth naming twice: a *shared external resource* — a browser profile, a DB, one API session — needs a lock of its own, keyed to the resource, not the runner; §4.)

What this guide is deliberately *not*, so you know its edges: `[BEYOND-LLM: proprietary-data — honest limits]`

- **Not a hard dollar cap.** That's a workspace spend limit or a boundary hook (§3). This makes spend loud and bounded, not impossible.
- **Not a security boundary.** Least-privilege and a hook reduce blast radius; they don't sandbox a compromised runner. Run untrusted work in a container/VM as a low-privilege user.
- **Not a scheduler guarantee.** A userland lock is defeated if the *wrapper itself* is `kill −9`'d before the reclaim logic runs, or on power loss. Belt-and-suspenders is the scheduler's single-instance *plus* the hybrid lock, not either alone.

- **Not version-stable.** `claude -p` flags and JSON fields shift across CLI versions (the envelope in §3 is `claude 2.1.209`). Pin your CLI version and re-test the wrapper on upgrade; treat `--output-format json` field names as a contract that can change.
- **Not uniformly applied even by us.** The page-2 audit is the proof: 4 of our unattended runners still lack a hard timeout. We ship the auditor *because* we drift too. Our minimal consult runner (`consult-twin.sh`) omits the timeout *on purpose* — it's foreground and human-triggered, so a hang is visible. Harden *unattended* runners; don't over-engineer a tool a human is watching.
- **Tools lie too — including ours.** The auditor in §9 is a grep heuristic: it caught our real gaps, but its first version silently passed a broken check because a `--`-leading pattern was swallowed by `grep` as an option (fixed 2026-07-27, noted in the script). A heuristic auditor flags what's *absent*; it can't judge whether what's *present* is correct (it counts a racy PID-file lock as "has a lock"). Read the ⚠️ cells by hand.
- **Mac-flavored, Linux-equivalent-inline.** Our idioms are `perl-alarm`, `mkdir`, `launchctl`, BSD `stat`. Every one has its Linux equivalent in-line above (`timeout`, `flock` / `mkdir`, `systemd`, `stat -c`). None of the *lessons* are stack-gated; only the spellings are.

---

§9 — The kit: harden your runner today

1. Audit what you already run — start here. `preflight-audit.sh` (bundled) reads a runner — or a whole directory — and scores it against the six deaths, printing the one-line fix for each gap. It's the same tool that produced the page-2 table. Run it before you read the rest of the kit; it tells you which sections you actually need. `[BEYOND-LLM: decision-tool — executed this build across 11 runners, receipts/2026-07-27-fleet-audit.txt]`

```
bash preflight-audit.sh ~/bin/my-claude-job.sh      # one runner
bash preflight-audit.sh --fleet ~/bin               # every *.sh in a dir; exit
        code = #runners with gaps
```

2. The template. Copy the Page 1 template. Replace the prompt and `--allowedTools`. That's the six guards — now including the kill-escalating timeout, the leak-proof hybrid lock, and the missing-state assert — in ~50 lines.

3. The six-death pre-flight checklist. Before you walk away from a scheduled runner, confirm all six:

- [ ] **Hang** → a hard `timeout` / `perl-alarm` at ~2× p99 runtime wraps the `claude` call **and escalates to `SIGKILL`** (`--kill-after`).

- [ ] **Silent burn** → `--max-turns` + a pinned cheap model + an ALARM on empty/error/ `is_error:true` + a workspace spend cap for the hard ceiling.
- [ ] **Double-fire** → an atomic `mkdir` lock **with PID/age reclaim** (survives `kill -9`) *and* the scheduler's single-instance; a lock for any *shared* resource too.
- [ ] **State-bleed** → prompt composed from files **and asserted complete** (`case *"(MISSING:"*`), run in `mktemp -d` ; state persisted atomically (write-temp-then-rename), idempotent so a double-fire is harmless.
- [ ] **Permission-block** → tools allow-listed *or* skip-permissions inside a sandbox + hook. Assert an expected marker in the output; branch on `is_error` , not just `$?` .
- [ ] **Drift** → one shared skeleton, model/effort as env knobs, one matrix row per runner — and re-run `preflight-audit.sh` on the fleet monthly.

4. The scheduler-poison lookup. launchd/systemd → the **timeout** is load-bearing (overlap-refusal makes a hang a silent skip). cron/k8s → the **lock** is load-bearing (overlap allowed → double-fire). Add `concurrencyPolicy: Forbid` (k8s) or a `concurrency:` group (GitHub Actions).

5. The "is my runner loud when it dies?" test. Answer all three or it can still run dark:

- If it **hangs**, what kills it, and after how long? *(timeout + kill-escalation)*
- If it **errors or returns empty**, who gets paged? *(in-run ALARM + `is_error` branch)*
- If it **never starts, or the box — or the whole fleet — is down**, what notices? *(a watchdog + a dead-man's switch that lives on someone else's infra)*

6. The model/effort matrix (§6) — fill one row per runner; default `haiku` unless quality pays for itself.

Everything in this kit was executed during this guide's build. The runnable artifacts are bundled in `receipts/` : `preflight-audit.sh` (the auditor) + `2026-07-27-fleet-audit.txt` (the real fleet scan); `demo-harness.sh` + `2026-07-22-demo-output.txt` (the six wrapper demos); `variant-demos.sh` + `2026-07-27-variant-output.txt` (the four day-2 variant paths); `capture-envelope.sh` + `2026-07-27-json-envelope.txt` (a live `claude -p` JSON envelope); and `2026-07-22-artifacts.md` (the raw fleet scripts).

Receipts: `chesky-autopost.sh` (25h-hang timeout + loud alarms, dated 2026-07-11) · `guide-build.sh` (mkdir lock+trap, six-file prompt composition, mktemp isolation, env tiering) · `chesky-rail-watchdog.sh` (STALL/SILENT watchdog, the week's post-mortem in its header) · `chesky-eod-report.sh` ("a zero is the headline") · `chesky-drift-heal.sh` + `consult-twin.sh` + the guide-factory rails (the fleet audit) · operating-log rows 2026-07-11, 07-14, and the 07-13→15 fleet-dark review · six wrapper demos (2026-07-22) + four variant-path demos +

one live JSON envelope (2026-07-27). Numbers marked [S] are imported as reported, not verified by us.
